

Week - 04 - Exceptions, Dataflow\Lecture_8_code_2.py

```

1  m = [
2      [1,   3,   None, 4,   None, None, 7,   None],
3      [None, None, 3,   None, 5,   None, None, 8   ],
4      [None, 2,   None, None, None, 6,   None, None],
5      [9,   None, None, 1,   None, None, 4,   None],
6      [None, None, 7,   None, 2,   None, None, 5   ],
7      [None, 6,   None, 8,   None, 3,   None, None],
8      [8,   None, None, 2,   None, None, 9,   None],
9      [None, 4,   None, None, None, 7,   None, 1   ],
10 ]
11
12 def is_group_complete(m, gi, gj):
13
14     # is_row_index_valid
15
16     # if gi+2 <= len(m):
17     #     is_row_index_valid = True
18     # else:
19     #     is_row_index_valid = False
20
21     is_row_valid = gi+2 <= len(m)
22     is_col_valid = gj+2 <= len(m[0])
23
24     return is_row_valid and is_col_valid
25
26     # return gi+2 <= len(m) and gj+2 <= len[m[0]]
27
28 # print(is_group_complete(m, 6, 6))
29
30 def is_single_group(m, gi, gj):
31     count = 0
32     for i in range(gi, gi+2):
33         for j in range(gj, gj+2):
34             if m[i][j] is not None:
35                 count = count + 1
36     if count == 1:
37         return True
38     else:
39         return False
40
41 # print(is_single_group(m, 1, 0))
42
43
44 def rotate_group(m, gi, gj):
45
46     m[gi][gj], m[gi+1][gj], m[gi][gj+1], m[gi+1][gj+1] = \
47     m[gi+1][gj], m[gi][gj], m[gi+1][gj+1], m[gi][gj+1]
48
49 rotate_group(m, 0, 0)
50 for line in m:
51     print (line)

```

```
52 |  
53 |  
54 | def iteration(n, m):  
55 |     pass  
56 |  
57 |
```